Final Project: Pedalboard

Jimena Arruti - 250290109

Introduction

This project consisted of the implementation of a Pedalboard VST and Standalone plugin using JUCE. The Pedalboard is composed of an octave-down effect, an overdrive effect, and a reverb that includes freeze and shimmer options. All effects can be used simultaneously without generating audible artefacts, and each pedal can be bypassed.

Aim and motivation

As an electric guitarist, I have my own pedalboard of effects that I use while playing, which is shown in Figure 1. While I chose each one of those particular effect pedals for its expressive capabilities and variety of achievable timbre, transporting the pedalboard can sometimes be troublesome. This is particularly true when I travel by plane, and I don't have space for an extra bag with considerable weight.

With that in mind, I aim to develop a virtual approximation of my pedalboard, consisting of an octaver (which I usually use to turn the signal one octave down), an overdrive and a reverb. This would permit me to achieve similar sounds while playing, using an interface and my computer instead of the pedalboard. Ideally, by using the plugin in standalone mode, the usage would be very straightforward, without needing a DAW.

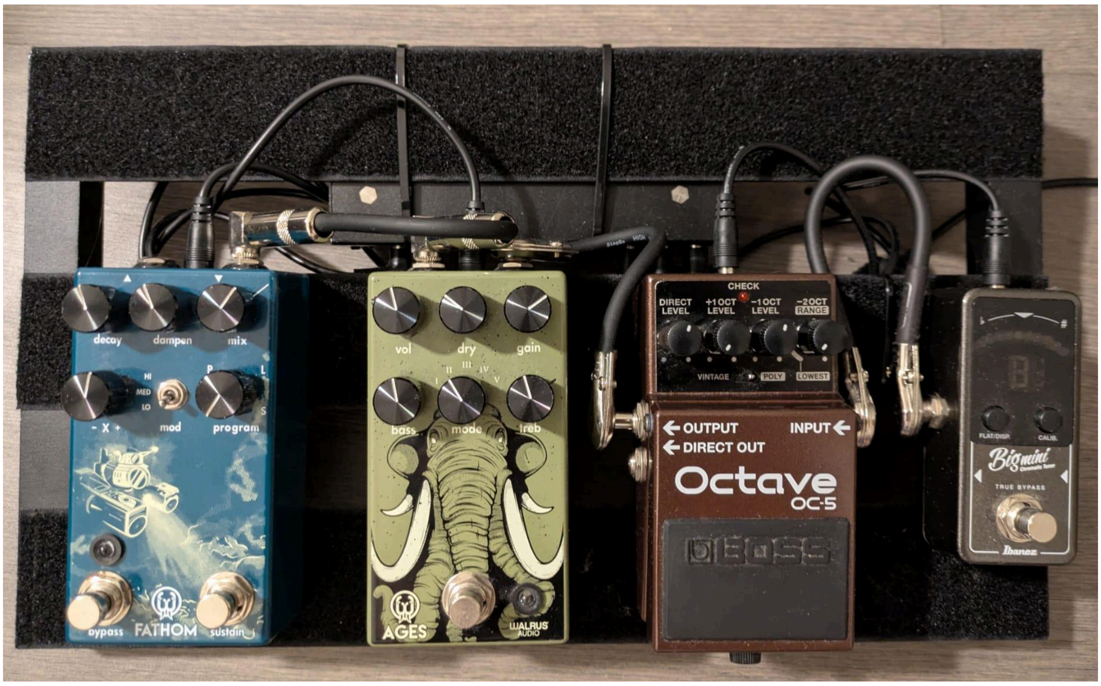


Figure 1: My current hardware pedalboard (reverb, overdrive, octaver and tuner)

Related work

There are several plugins that offer simulations of guitar gear, including amps and effect pedals. Such is the case of IK Multimedia's Amplitube[1] or Native Instruments' Guitar Rig[2]. Additionally, Logic Pro includes a Pedalboard effect, but it is a software product that only runs on macOS. The aesthetic of these plugins served as an inspiration for the design of the GUI.

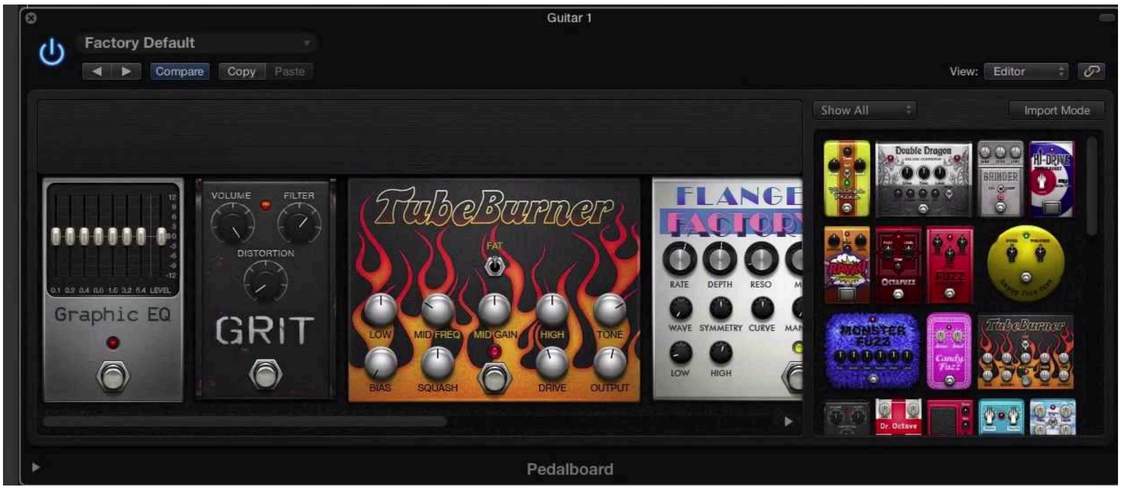

Guitar 1
Factory Default
Compare
Copy
Paste
View:
Editor
Show All
Import Mode
Graphic EQ
VOLUME
FILTER
DISTORTION
GRIT
TubeBurner
FAT
LOW
MID FREQ
MID GAIN
HIGH
TONE
BIAS
SQUASH
DRIVE
OUTPUT
FLANGE FACTORY
RATE
DEPTH
RESO
WAVE
SYMMETRY
CURVE
LOW
HIGH
Double Dragon
GRINDER
HI-DRIVE
OCTAFUZZ
FUZZ
MONSTER FUZZ
Candy Fuzz
Dr. Octave
Pedalboard

Figure 2: Logic Pro Pedalboard[3]

Methodology

The plugin was developed using an iterative design workflow. This applied both to the digital signal processing and the graphical interface, making the testing of new functionalities manageable.

The starting point was the code for creating distortion and reverb effects seen in the module during lab 10[4], for which a GUI was created resembling two guitar pedals. Note that while in the hardware pedalboards the pedals are ordered from right to left, in this GUI, the order is from left to right, as expected in a computer workflow. The customisation of the look and feel was guided first by JUCE's tutorial on varying look and feel in a plugin[5], and later by consulting forums and interacting with Gemini AI. A free-to-use image strip[6] was used to get a realistic visual of knobs.

The original code implemented distortion with five different algorithms while permitting the variation of a gain parameter, and used JUCE's reverb with exposed room size, damping, width, wet and dry controls. The signal processing changes at this stage were: the addition of the dry/wet control and level knobs for distortion, whose effects were manually applied in the code; the unification of the dry and wet parameters in a single dry/wet control for the reverb; and the addition of freeze functionality of the reverb which was an existing parameter of JUCE's implementation. Once the two blocks were working, additional attention was given to the sound of the distortion, keeping two of the existing algorithms (hard clipping and soft clipping) and adding the hyperbolic tangent algorithm as an option.

---

[1] https://www.ikmultimedia.com/products/amplitube5cs/

[2] https://www.native-instruments.com/en/products/komplete/guitar/guitar-rig-7-player

[3] Image from: https://www.youtube.com/watch?v=QDdbrQuXW9Y

[4] https://github.com/nellyngz95/JuceSteps/tree/main/ReverbAndDistortion

[5] https://juce.com/tutorials/tutorial_look_and_feel_customisation/

[6] https://forum.hise.audio/topic/6427/free-hise-filmstrips-analog-knob-kit-01-by-noisehead

After the reverb and distortion (later renamed overdrive) pedal were working, the design proceeded with the implementation of a pitch shifter based on the phase vocoder. Since the concept was complex, and the code was meant to be reutilised inside the reverb pedal to obtain the shimmer effect, it was decided to create a class to encapsulate functionality and state, therefore permitting the creation of more than one instance in a tidy manner. The code was inspired by the course book[1] and the implementation by Juan de Gil available on the course repository[7], and was simplified to work for a fixed ratio, since it was intended to only alter the signal by transposing an octave down (in the octave down pedal) or an octave up (in the shimmer effect). The values of *window_size=2018* and *overlap=8* were selected. Once the pitch shifter was working, the dry/wet control was added.

After the three main blocks were working, it was time to add the shimmer effect to the reverb pedal. At this point, it was clear that the code should be further modularised, and the reverb effect should also be implemented in a class. The shimmer effect was added inside the *PedalReverb* class, as the possibility of octave the reverb tail obtained by JUCE's implementation. Further changes needed to be done to achieve this effect, as externalising the dry/wet control to permit the pitch shift of the reverb tail before doing the mix.

When testing the plugin, two main defects became audible. The first was that audio crackling was present when the three pedals were active, and in particular, when the shimmer of the reverb was also on. It was evident that the code needed to be optimised to consume less CPU, which was done by moving heap allocations out of processing loops. Tests were conducted to reduce the value of some parameters of the pitch shifter, in particular the *window size* and the *overlap* values, but the resulting sound was muddier (probably due to the lesser frequency resolution, and the smearing that a higher *overlap* softens). In the end, the decision was made to turn the plugin into mono only, which eventually freed up resources, while still serving the original purpose of processing a guitar input. The second was that whenever the on/off toggles were used, pops appeared. To solve this issue, the binary logic tied to the toggle was replaced by a crossfade between the dry signal and the effected, replicating a true bypass behaviour. Since keeping internal state is necessary for that task, the distortion code was also encapsulated in a class. Finally, for additional safety, the internal buffers of the pitch shift objects were set to be cleared when the effect was finally silenced.

---

[7] https://github.com/joshreiss/Audio_Effects-Theory_Implementation_and_Application/tree/main/10%20Phase%20vocoder/Source%20code/Pitch%20Shift%20-%20JDGC

Flow diagram

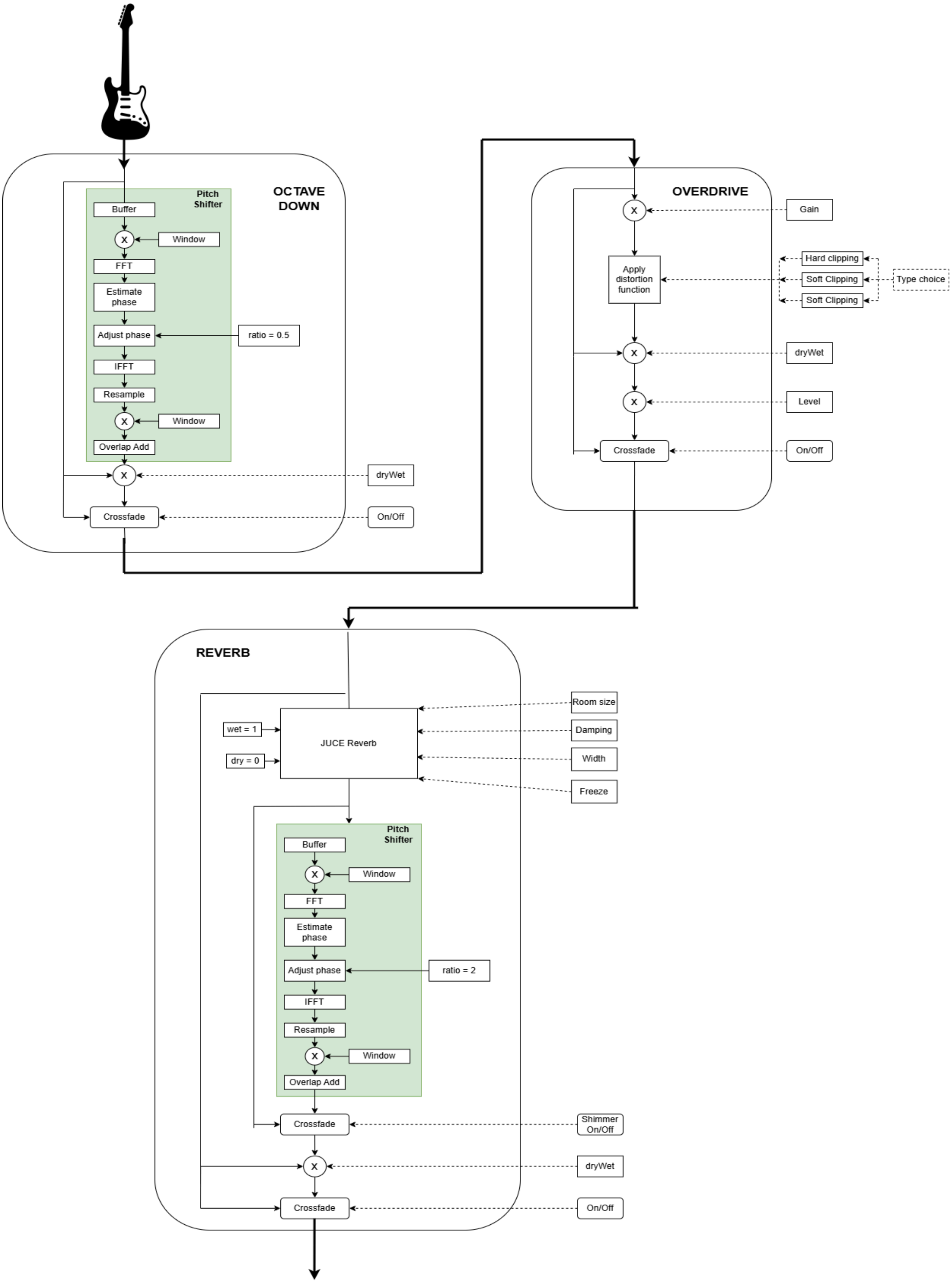
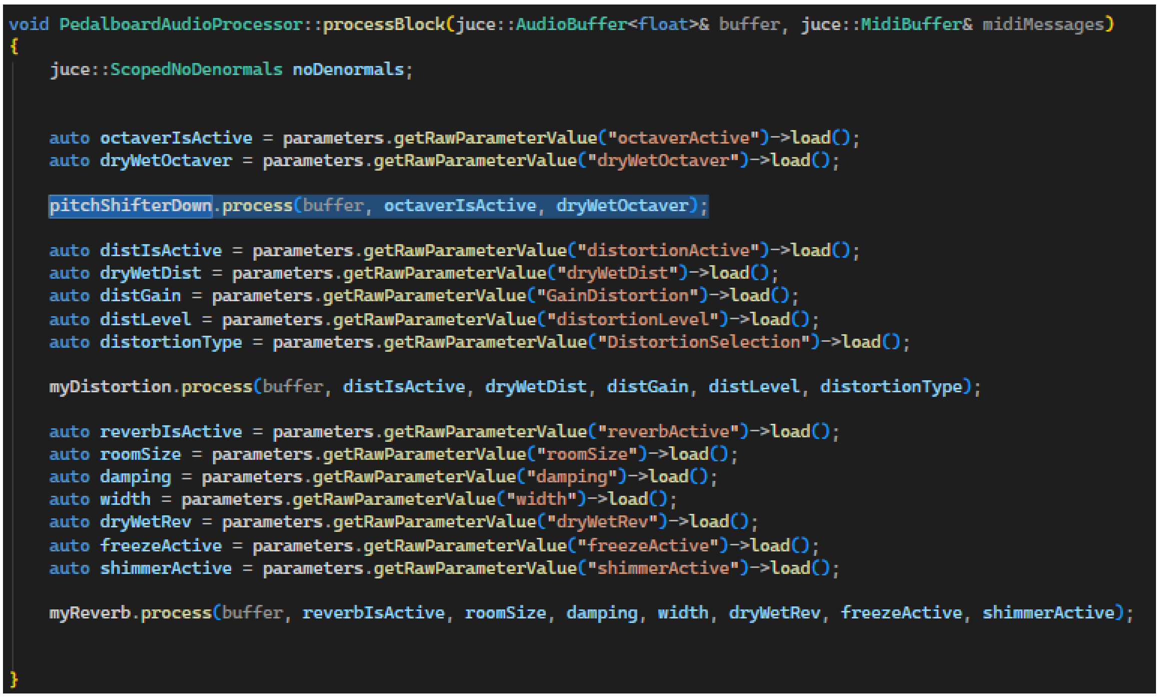

OCTAVE DOWN
Pitch Shifter
Buffer
X
Window
FFT
Estimate phase
Adjust phase
ratio = 0.5
IFFT
Resample
X
Window
Overlap Add
X
dryWet
Crossfade
On/Off
OVERDRIVE
X
Gain
Apply distortion function
Hard clipping
Soft Clipping
Soft Clipping
Type choice
X
dryWet
X
Level
Crossfade
On/Off
REVERB
wet = 1
dry = 0
JUCE Reverb
Room size
Damping
Width
Freeze
Pitch Shifter
Buffer
X
Window
FFT
Estimate phase
Adjust phase
ratio = 2
IFFT
Resample
X
Window
Overlap Add
Crossfade
Shimmer On/Off
X
dryWet
Crossfade
On/Off

Figure 2: Flow Diagram

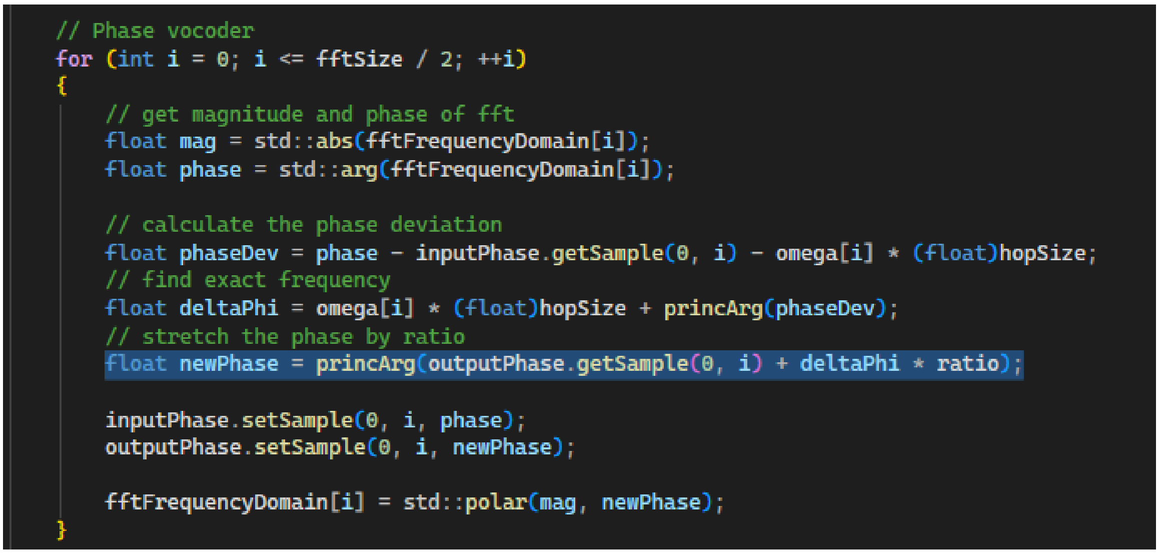
Code

The code is structured using three custom classes, one for each Pedal. This makes the code easier to understand, since the *processBlock* calls the process method of every pedal separately.



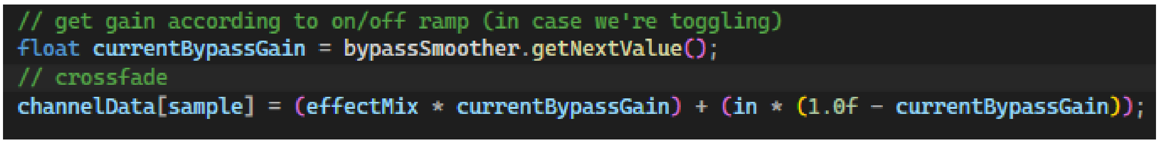
```
void PedalboardAudioProcessor::processBlock(juce::AudioBuffer<float>& buffer, juce::MidiBuffer& midiMessages)
{
    juce::ScopedNoDenormals noDenormals;


    auto octaverIsActive = parameters.getRawParameterValue("octaverActive")->load();
    auto dryWetOctaver = parameters.getRawParameterValue("dryWetOctaver")->load();

    pitchShifterDown.process(buffer, octaverIsActive, dryWetOctaver);

    auto distIsActive = parameters.getRawParameterValue("distortionActive")->load();
    auto dryWetDist = parameters.getRawParameterValue("dryWetDist")->load();
    auto distGain = parameters.getRawParameterValue("GainDistortion")->load();
    auto distLevel = parameters.getRawParameterValue("distortionLevel")->load();
    auto distortionType = parameters.getRawParameterValue("DistortionSelection")->load();

    myDistortion.process(buffer, distIsActive, dryWetDist, distGain, distLevel, distortionType);

    auto reverbIsActive = parameters.getRawParameterValue("reverbActive")->load();
    auto roomSize = parameters.getRawParameterValue("roomSize")->load();
    auto damping = parameters.getRawParameterValue("damping")->load();
    auto width = parameters.getRawParameterValue("width")->load();
    auto dryWetRev = parameters.getRawParameterValue("dryWetRev")->load();
    auto freezeActive = parameters.getRawParameterValue("freezeActive")->load();
    auto shimmerActive = parameters.getRawParameterValue("shimmerActive")->load();

    myReverb.process(buffer, reverbIsActive, roomSize, damping, width, dryWetRev, freezeActive, shimmerActive);


}
```

The code of the octaver is of particular interest, since it makes use of the phase vocoder algorithm. This algorithm processes the signal in buffered windows, whose processed outputs are later added together via the Overlap-Add method. In the snippet below, we can see the processing that occurs in the frequency domain for one window, where for each frequency bin, a new phase is calculated according to the ratio (which is 0.5 or 2 for the octave down and shimmer, respectively).



```
// Phase vocoder
for (int i = 0; i <= fftSize / 2; ++i)
{
    // get magnitude and phase of fft
    float mag = std::abs(fftFrequencyDomain[i]);
    float phase = std::arg(fftFrequencyDomain[i]);

    // calculate the phase deviation
    float phaseDev = phase - inputPhase.getSample(0, i) - omega[i] * (float)hopSize;
    // find exact frequency
    float deltaPhi = omega[i] * (float)hopSize + princArg(phaseDev);
    // stretch the phase by ratio
    float newPhase = princArg(outputPhase.getSample(0, i) + deltaPhi * ratio);

    inputPhase.setSample(0, i, phase);
    outputPhase.setSample(0, i, newPhase);

    fftFrequencyDomain[i] = std::polar(mag, newPhase);
}
```

Finally, the processing methods of all three classes include a similar crossfade to turn the effect on and off without generating clicks.



```
// get gain according to on/off ramp (in case we're toggling)
float currentBypassGain = bypassSmoother.getNextValue();
// crossfade
channelData[sample] = (effectMix * currentBypassGain) + (in * (1.0f - currentBypassGain));
```

Analysis

The outputs from two possible plugin configurations were analysed, contrasting their waveforms and spectrograms with those of the input.

Octaver

The spectrogram for the original signal can be seen in Figure 3. We can clearly observe four distinct blocks corresponding to the four arpeggiated chords, and for each one, we can see each attack note and its corresponding harmonics. It’s worth noting that there is a high overlap of harmonics, as the notes are sustained while the chord is present. For the purposes of understanding the effect of the octaver, which is supposed to transpose the input one octave down, it’s useful to focus on single notes at a time. Looking at the first note, we can see it corresponds approximately to 194Hz.

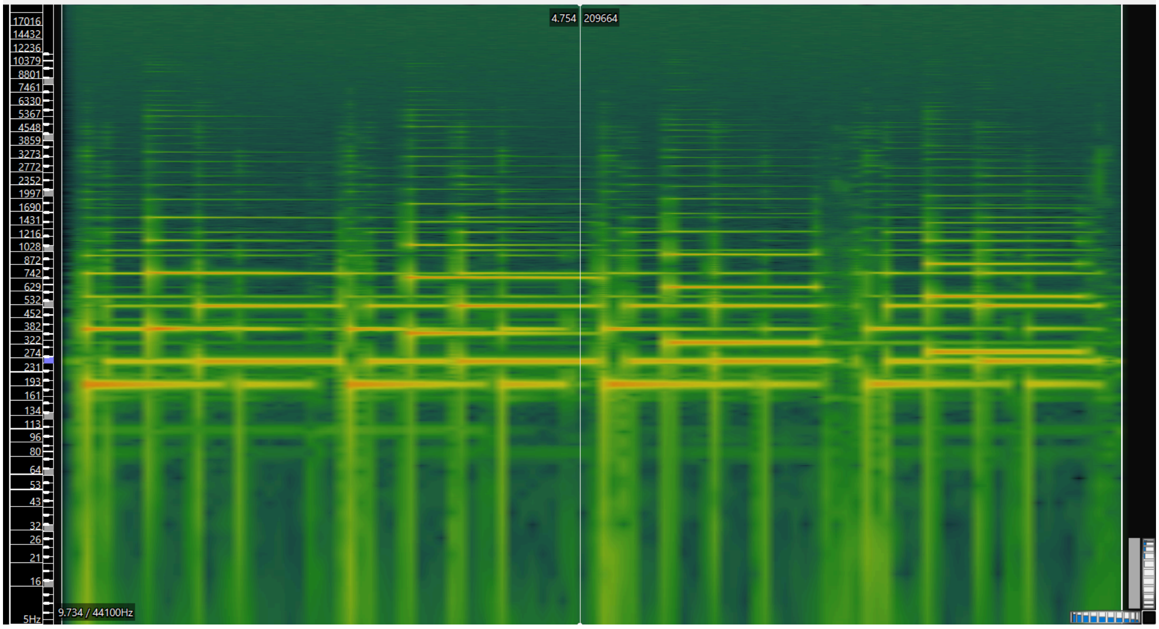


Figure 3: Spectrogram of input signal for an 8192 window length and 50% overlap

When contrasting this to the spectrogram of the octave-down signal, as shown in Figure 4, it is evident that the general structure of the signal is maintained while the frequency content is now one octave lower. This is confirmed by the first note now being centred at 97Hz.

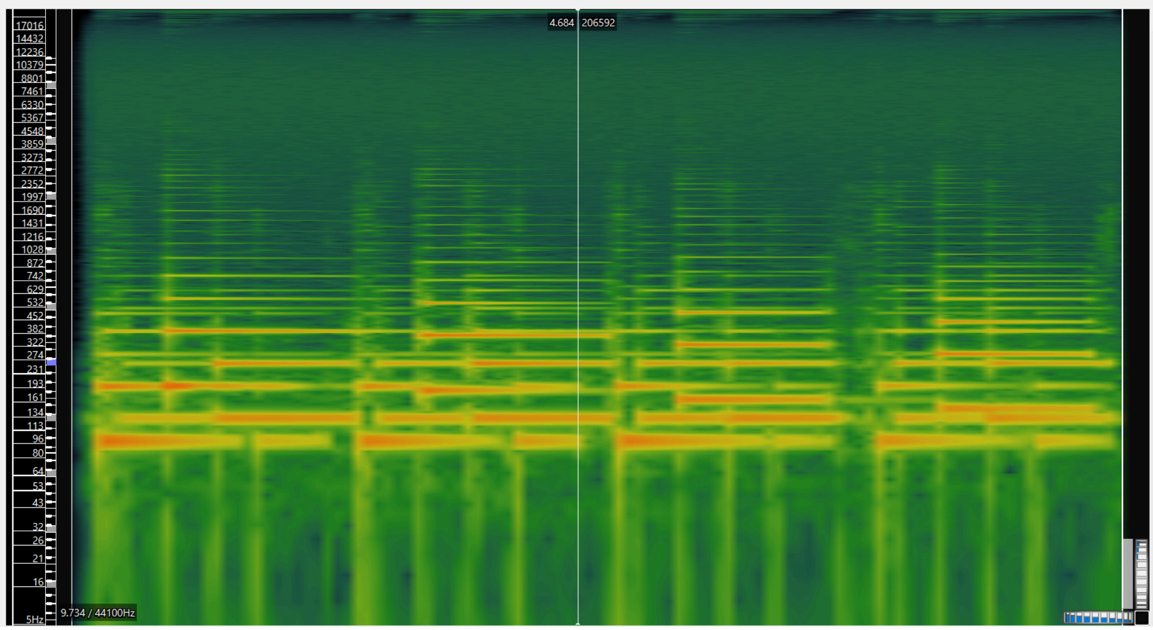


Figure 4: Spectrogram of the octaved signal for an 8192 window length and 50% overlap

Hard-clipping distortion

The waveform for the original signal can be seen in Figure 5, while the waveform for the distorted signal can be seen in Figure 6. The signal was distorted using the hard clipping algorithm, with Gain set to 10, dryWet set to 1 (fully wet), and with Level set to 0.4 to match perceived loudness.

It is clear that hard clipping distortion is working as expected, since the amplitude of the output waveform is 0.2, which is exactly the value we get when the signal is hardclipped (0.5) multiplied by the Level of 0.4. Additionally, Figure 7 shows the spectrogram of the distorted signal, where a greater number of harmonics is observed, which is characteristic of distortion.

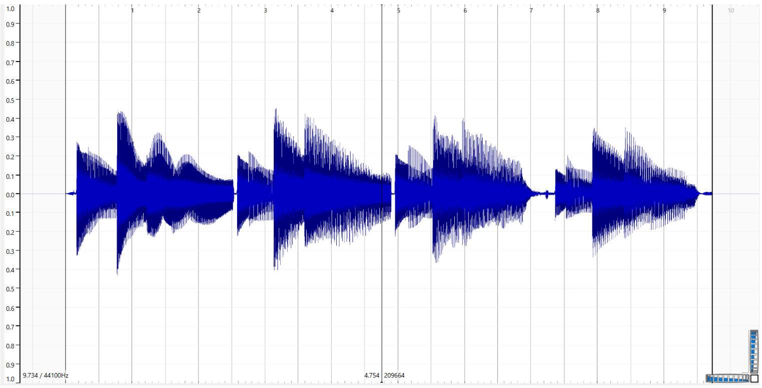


Figure 5: Waveform of input signal

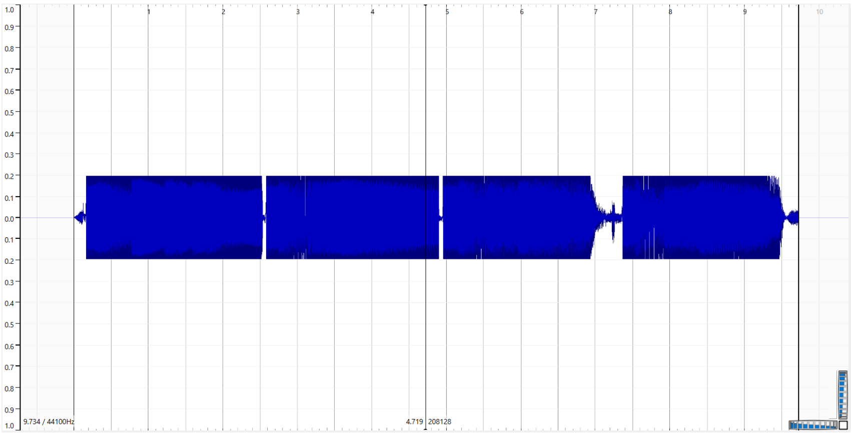


Figure 6: Waveform of hard-clipped signal, levelled to match loudness

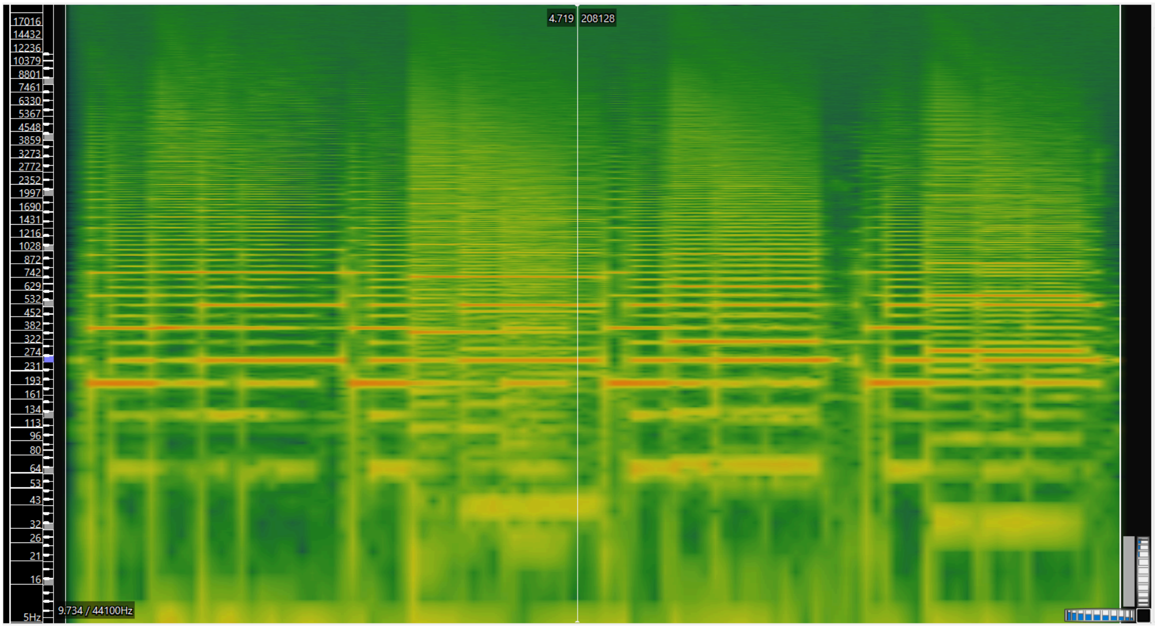
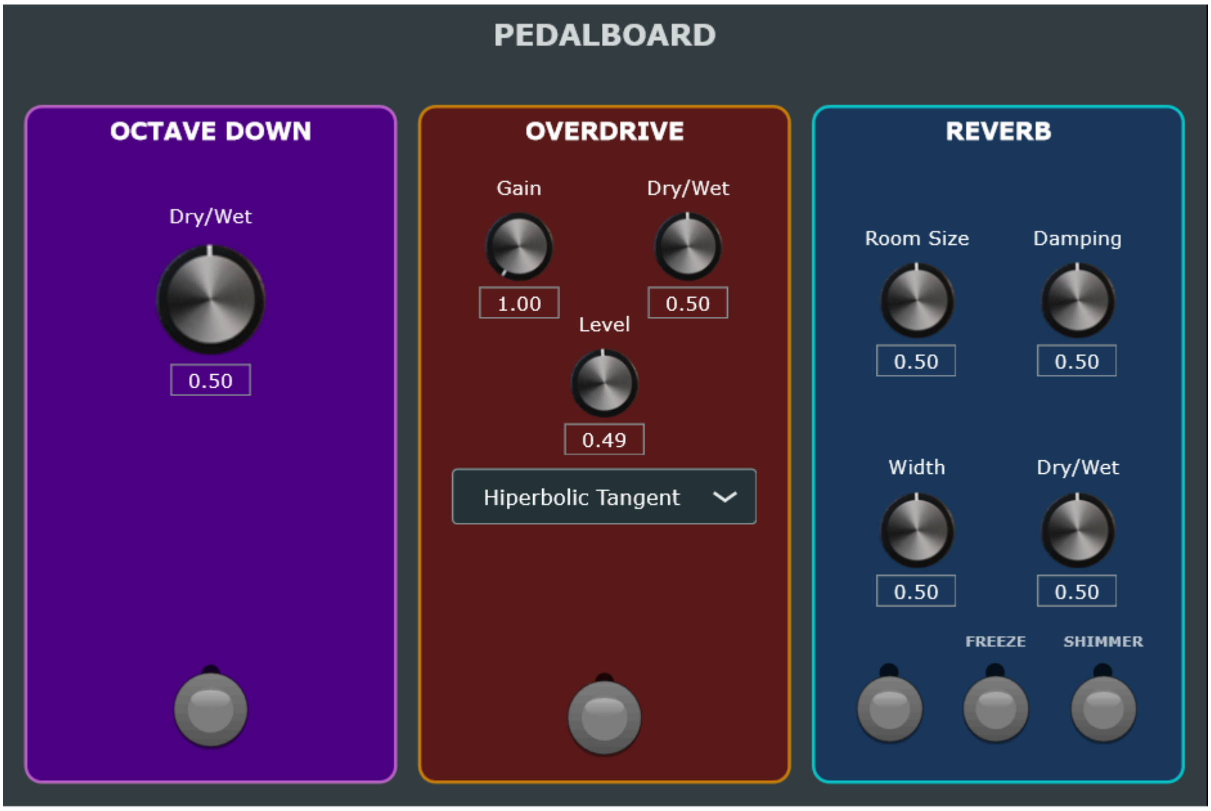


Figure 7: Spectrogram of hard clipped signal, leveled to match loudness, 8192 window length and 50% overlap

Results

A VST3 and Standalone plugin was developed to mimic the basic functionalities of an existing pedalboard. It includes an octave down, overdrive and reverb submodules, correctly supporting true bypass in each one. The resulting GUI is aesthetically pleasing, including realistic knobs and graphically detailed stomp switches with their corresponding LEDs indicating status, as can be seen in Figure 8.

By designing and implementing this project, I was able to better understand the concepts behind the module, in particular, the application of phase vocoders for pitch shifting. Additionally, the size of the project inspired me to better organise my code, implementing classes to better encapsulate code and maintain state. Finally, the amount of processing needed by all the effects operating simultaneously forced me to dive deeper into the algorithms, seeking to optimise CPU.



PEDALBOARD
OCTAVE DOWN
OVERDRIVE
REVERB
FREEZE
SHIMMER

Dry/Wet 0.50

Gain 1.00
Dry/Wet 0.50
Level 0.49
Hiperbolic Tangent

Room Size 0.50
Damping 0.50
Width 0.50
Dry/Wet 0.50

Figure 8: Plugin GUI

References

[1] Reiss, J.D., & McPherson, A.P. (2026). Audio Effects: Theory, Implementation and Application (2nd ed.). Routledge. https://doi.org/10.1201/9781003593942